UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISION DE CIENCIAS DE LA INGENIERÍA

SISTEMAS DE BASES DE DATOS 2
Cátedra: Ing. Daniel Gonzales
Segundo Semestre de 2026

PROYECTO 2 — USAC ANALYTICS

Informe de desarrollo, integración y validación de MongoDB y Neo4j

ESTUDIANTE
Byron Fernando Torres Ajxup
Carné: 201731523

Quetzaltenango, octubre de 2026

Resumen Ejecutivo

El presente informe documenta el desarrollo, implementación y evaluación del proyecto USAC Analytics, una solución web e interactiva orientada al análisis avanzado de las pruebas específicas de la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala (USAC). A través de la integración híbrida de dos modelos NoSQL documental con MongoDB y de grafos con Neo4j, junto con una arquitectura moderna compuesta por FastAPI en el backend y React en el frontend, el sistema procesa exitosamente un historial multianual de 208,453 evaluaciones correspondientes a 42,371 aspirantes únicos entre los años 2015 y 2024. El informe detalla de manera exhaustiva el flujo ETL, el diseño y normalización de esquemas, las 20 consultas agregadas en MongoDB y 5 en Cypher (Neo4j), los hallazgos analíticos, la comparativa técnica entre motores NoSQL, los manuales técnico y de usuario, y las principales conclusiones del sistema.

Objetivos

Objetivo General

Integrar dos paradigmas de bases de datos NoSQL para investigar los resultados de las evaluaciones de ingreso a la facultad.

Objetivos Específicos

Normalizar y validar los archivos CSV provistos, garantizando la carga de datos sin duplicaciones al reimportar archivos. Asimismo, resolver veinte consultas en MongoDB y cinco consultas Cypher en Neo4j, construir una API REST explicativa, exponer un panel de control (dashboard), buscador, explorador de grafos y módulo de carga CSV, registrando métricas de rechazos y tiempos de ejecución.

Fuente de Datos

Los datos provienen del portal oficial de datos abiertos: https://datosabiertos.ingenieria.usac.edu.gt/en/dataset/pruebas-especificas

Los archivos CSV incluyen los siguientes campos: fecha_asignacion, sexo, anio_nacimiento, materia, numero_de_fecha_de_evaluacion, anio_de_ingreso, aprobacion, carrera_objetivo, departamento_institucion_educativa, municipio_institucion_educativa, tipo_institucion_educativa y correlativo_aspirante. El correlativo identifica de forma anónima al aspirante, quien puede contar con múltiples evaluaciones.

Arquitectura del Sistema

El sistema se encuentra desacoplado en tres capas principales:

- **Frontend:** Construido con React, TypeScript, Vite y Tailwind CSS. Utiliza Recharts para visualizaciones analíticas y Cytoscape.js para la renderización de grafos.
- **Backend:** Desarrollado en Python con FastAPI, actuando como controlador API REST y gestor de servicios de consulta.
- **Almacenamiento Persistente:** Usa MongoDB para la gestión documental de evaluaciones y Neo4j para el modelado explícito de relaciones. La comunicación sigue el flujo Navegador → FastAPI → MongoDB/Neo4j, manteniendo las credenciales bajo variables de entorno en un archivo privado .env.

Proceso ETL (Extracción, Transformación y Carga)

El archivo CSV se lee utilizando la codificación UTF-8-SIG y se estandarizan los encabezados abreviados presentes en algunos periodos. Durante el proceso, se convierten las fechas, los enteros y el indicador booleano de aprobación, calculando la edad aproximada únicamente cuando existe un año de nacimiento válido. Se omiten y descartan las filas sin campos obligatorios, con números de intento menores a 1 o con estados de aprobación distintos de APROBADO o NO APROBADO. Para la persistencia, MongoDB procesa lotes de 500 registros realizando operaciones de upsert basadas en el hash SHA-256 (nombre_archivo:fila). En Neo4j se implementan cláusulas MERGE para nodos y relaciones. Finalmente, el sistema audita registros válidos, rechazados, motivos de falla y tiempos de ejecución.

Modelado de Datos

En MongoDB, la colección principal es *evaluaciones*, acompañada por las colecciones de soporte *historial_cargas, errores_carga* y *resumen_carrera*. Por su parte, Neo4j organiza la información a través de las etiquetas *Aspirante, Evaluacion, Materia, Carrera, InstitucionTipo, Municipio, Departamento* y *PeriodoIngreso*; vinculadas mediante las relaciones *REALIZO, EVALUA, PARA_CARRERA, PROVIENE_DE_TIPO, PROCEDE_DE, PERTENECE_A* y *EN_PERIODO*.

Los conteos globales verificados en el grafo de Neo4j comprenden: 42,371 nodos Aspirante, 208,453 Evaluacion, 3 Materia, 15 Carrera, 5 InstitucionTipo, 312 Municipio, 24 Departamento y 10 PeriodoIngreso.

Consultas Implementadas

MongoDB

Se implementaron 20 consultas agregadas que cubren aspectos analíticos clave: distribución de aspirantes por institución, aprobados por materia, aprobados por carrera y año, tasa de aprobación por materia, promedio de edad por carrera, generación de colecciones resumidas, distribución de edad de aprobados por carrera e institución, aprobados por municipio/carrera, evaluaciones mensuales en escuelas públicas, principales carreras demandadas por el rango de edad 16–18, historial de desempeño, distribución por sexo, tasa de aprobación por edad, intentos por materia, historial de evaluaciones completo, reprobados en el primer intento, principales municipios de procedencia, tasa por institución, demanda de carreras por departamento y evolución histórica anual.

Neo4j (Cypher)

Se construyeron 5 consultas estructuradas en Cypher: aprobaciones por materia, aspirantes por departamento, historial completo por correlativo, carreras más demandadas por departamento y rendimiento por número de intento. Todas se ejecutan desde el cliente web invocando las funciones encapsuladas en el módulo de servicios del backend.

Detalle de las 20 Consultas de MongoDB

1. Distribución de aspirantes por tipo de institución: Agrupa las evaluaciones según el origen educativo (PÚBLICO, PRIVADO, DIVERSIFICADO, etc.) calculando totales e índices de participación.

2. Aprobados por materia: Mide la cantidad de evaluaciones con estado APROBADO segregadas por cada asignatura evaluada (Matemática, Computación, etc.).

3. Aprobados por carrera y año: Matriz analítica que contabiliza aspirantes admitidos por combinación de carrera destino y período lectivo.

4. Tasa de aprobación por materia: Relación porcentual entre exámenes aprobados y exámenes totales sustentados para cada asignatura.

5. Promedio de edad por carrera: Cálculo de la media de edad de los aspirantes filtrados por la carrera objetivo seleccionada.

6. Colecciones resumidas por carrera: Generación e inserción de resúmenes consolidados a la colección de soporte *resumen_carrera* mediante pipelines de agregación $out.

7. Distribución de edad de aprobados por carrera e institución: Mapeo multidimensional que cruza la edad, el tipo de colegio y la carrera alcanzada.

8. Aprobados por municipio y carrera: Desglose de admisiones según el origen municipal del centro educativo de procedencia.

9. Evaluaciones mensuales en escuelas públicas: Serie temporal que contabiliza la concurrencia de evaluaciones aplicadas a estudiantes del sector público.

10. Principales carreras demandadas en el rango 16–18 años: Ranking de preferencia académica enfocado en el segmento demográfico recién egresado del nivel medio.

11. Historial de desempeño del aspirante: Seguimiento secuencial de los resultados de un estudiante específico ordenado por fecha e intento.

12. Distribución demográfica por sexo: Conteo y porcentaje de participación por género en el universo general de evaluaciones.

13. Tasa de aprobación por edad: Comportamiento de la efectividad en los exámenes agrupado en bloques etarios.

14. Intentos por materia: Promedio e índice de reincidencia de exámenes presentados por un aspirante antes de aprobar o abandonar una materia.

15. Historial de evaluaciones completo: Consulta detallada de registros consolidados con proyectores personalizados.

16. Reprobados en el primer intento: Identificación y cuantificación del volumen de aspirantes que no lograron la nota mínima en su primera evaluación.

17. Principales municipios de procedencia: Top geográfica de los municipios que más aportan aspirantes a la facultad.

18. Tasa de éxito por institución educativa: Comparativa de efectividad diagnóstica entre establecimientos públicos y privados.

19. Demanda de carreras por departamento: Distribución territorial de la preferencia de carreras a nivel departamental.

20. Evolución histórica anual: Tendencia de volumen de evaluados y tasa de aprobación en el período 2015–2024.

Detalle de las 5 Consultas Cypher (Neo4j)

1. Aprobaciones por materia: Recorre la red *(a:Aspirante)-[:REALIZO]->(e:Evaluacion)-[:EVALUA]->(m:Materia)* filtrando por estado APROBADO para computar los totales consolidados por asignatura.

2. Aspirantes por departamento: Conecta nodos de procedencia geográfica mediante *(e)-[:PROCEDE_DE]->(m:Municipio)-[:PERTENECE_A]->(d:Departamento)* para agrupar el flujo de aspirantes por región.

3. Historial completo por correlativo: Expande subgrafos individuales partiendo de un correlativo específico para extraer todas las evaluaciones, carreras aplicadas, materias sustentadas y resultados obtenidos.

4. Carreras más demandadas por departamento: Navega la relación tripartita entre origen departamental, evaluación y carrera elegida *(e)-[:PARA_CARRERA]->(c:Carrera)* identificando patrones de preferencia regional.

5. Rendimiento por número de intento: Analiza las relaciones con la propiedad de intento para medir la tasa de conversión diagnóstica a medida que los aspirantes reinciden en las evaluaciones.

Resultados y Análisis

Del total de 208,453 evaluaciones registradas, asociadas a 42,371 aspirantes únicos, la distribución por materia refleja los siguientes resultados:

- **MATEMATICA:** 124,427 evaluaciones, registrando 11,008 aprobaciones (8.85%).
- **COMPUTACION:** 84,004 evaluaciones, registrando 18,869 aprobaciones (22.46%).
- **MATEMATICA ITUGS:** 22 evaluaciones, registrando 0 aprobaciones (0.00%).

Es fundamental diferenciar el total de evaluaciones respecto al total de aspirantes únicos, dado que una misma persona puede contabilizar múltiples registros en función del número de intento, materia o año evaluado.

Pruebas y Evidencias

La prueba de reimportación con el dataset de 2015 procesó exitosamente 20,277 filas válidas sin rechazos, conservando el total de 208,453 registros originales. Adicionalmente, se ejecutó una prueba con un archivo de control de 4 filas que aceptó 1 registro y rechazó 3, explicitando las causas: resultado desconocido, intento no positivo y materia vacía. En la API REST se corrigió un problema de serialización asociado al tipo *ObjectId* de MongoDB, alcanzando respuestas satisfactorias HTTP 200 OK.

Las capturas explicativas de la interfaz gráfica y de las herramientas de administración se adjuntan en el anexo de evidencias.

Conclusiones

La combinación de MongoDB y Neo4j demostró ser altamente eficiente: MongoDB agiliza las agregaciones de datos masivos mientras que Neo4j expone con claridad las redes de relaciones entre aspirantes, asignaciones y dependencias geográficas o institucionales. La implementación del pipeline ETL garantiza la integridad de la información mediante procesos de validación estricta y auditoría de rechazos.

Limitaciones y Trabajo Futuro

Actualmente la carga hacia ambas bases de datos no opera bajo una transacción distribuida atómica. Asimismo, el identificador único no evita duplicados semánticos entre archivos distintos. Como trabajo futuro, se contempla implementar autenticación y controles de acceso, sincronización atómica entre motor documental y de grafos, y reglas analíticas para la detección avanzada de inconsistencias.

Metodología de Desarrollo

El proyecto se abordó mediante una secuencia de etapas verificables. En primer lugar se revisó la estructura de diez CSV y se identificaron las diferencias de encabezados entre períodos. Posteriormente se diseñó una representación documental para almacenar cada evaluación normalizada y una representación de grafo para relacionar cada evaluación con el aspirante, la materia, la carrera, el tipo de institución, el municipio, el departamento y el año de ingreso. Después se implementaron los servicios de consulta y sus rutas REST, y finalmente se integrated la interfaz web. Las comprobaciones se realizaron desde el backend, el navegador, MongoDB y Neo4j. Este enfoque permitió aislar los problemas de transformación, almacenamiento y presentación.

Preparación y Calidad de Datos

La lectura de archivos en UTF-8-SIG permite manejar la marca de orden de bytes sin incorporarla al primer encabezado. La normalización resuelve diferencias como *numero_de_fecha_de_evaluaci, departamento_institucion_ed* y *municipio_institucion_*, encontradas en algunos años. Para evitar inconsistencias, las fechas se convierten a valores de fecha, los años e intentos a enteros y el resultado de aprobación a un booleano adicional. Los campos indispensables se validan antes de guardar. Un año de nacimiento no numérico, por ejemplo "SIN REGISTRO", se representa como dato ausente y no se utiliza en el cálculo de edad. Esta política evita que los agregados de edad se contaminen con valores ficticios.

Trazabilidad y Tratamiento de Errores

Cada registro aceptado conserva *archivo_origen, fila_origen* e *id_registro*. Esto permite volver a ubicar la fila original cuando se identifica una anomalía. Los documentos rechazados no pasan a la colección principal y sus motivos se conservan en *errores_carga*. La aplicación muestra hasta veinte motivos en la respuesta de una importación; el total de rechazos se contabiliza por separado. Durante las pruebas apareció un error de serialización porque PyMongo agregaba identificadores *ObjectId* a los diccionarios de errores utilizados en la respuesta. Se resolvió insertando copias de esos diccionarios para conservar una estructura JSON serializable en la API.

Diseño de la API REST

La API REST construida con FastAPI encapsula la lógica de negocio, sirviendo como capa intermedia entre las solicitudes del cliente y los motores de base de datos NoSQL. La API está organizada modularmente en controladores y servicios de consulta:

- **Rutas de Gestión ETL:** Permiten la carga, validación, procesamiento por lotes e historial de auditoría de archivos CSV.
- **Rutas Analíticas MongoDB:** Exponen los 20 pipelines de agregación para el dashboard y los módulos de consulta masiva.

Rutas de Grafos Neo4j: Ejecutan las 5 consultas estructuradas en Cypher y suministran datos formateados en nodos y aristas para la visualización interactiva en Cytoscape.js.

Interfaz y Organización por Componentes

La interfaz separa las pantallas por responsabilidad: el dashboard presenta indicadores y gráficos; la página de consultas ejecuta análisis predefinidos de MongoDB; otra página permite las cinco consultas Cypher; el explorador dibuja nodos y aristas de aspirantes; el buscador combina filtros y paginación; el módulo de importación recibe CSV y muestra resultados por archivo. Esta separación facilita mantenimiento y pruebas. Para el explorador se utiliza Cytoscape.js, con colores diferenciados por entidad y controles para centrar, acercar, alejar y seleccionar nodos. Las gráficas de barras permiten comparar tasas de aprobación de las materias.

Indicadores y Criterios de Interpretación

El denominador de una tasa debe quedar explícito. Por ejemplo, el porcentaje de aprobación de Matemática se calculó sobre 124,427 evaluaciones, mientras que el de Computación se calculó sobre 84,004. No sería correcto comparar esos conteos con aspirantes únicos como si describieran la misma unidad de análisis. La consulta de desempeño por número de intento permite estudiar si varía la frecuencia de aprobación conforme aumenta el número de evaluación. Sin embargo, los datos no identifican necesariamente los factores que provocaron dichas variaciones. Las diferencias entre áreas geográficas o tipos de institución pueden reflejar composición y número de registros disponibles, por lo que deben interpretarse con cautela.

Comprobaciones de Integridad

La comprobación de integridad posterior a la importación confirmó 208,453 documentos en MongoDB y 208,453 nodos Evaluacion en Neo4j. Asimismo se verificaron 42,371 nodos Aspirante, 15 Carrera, 24 Departamento, 5 InstitucionTipo, 3 Materia, 312 Municipio y 10 PeriodoIngreso. Una consulta de Neo4j comprobó que las 208,453 evaluaciones tenían al menos una relación con un aspirante, una materia y una carrera. Una importación adicional con datos de prueba produjo cuatro filas procesadas, una aceptada y tres rechazadas; tras eliminar el registro temporal, ambos motores volvieron al total original.

Criterios de Reproducibilidad

La ejecución del proyecto requiere iniciar previamente MongoDB y Neo4j, establecer las credenciales y las URI en el archivo privado .env, instalar las dependencias del backend y del frontend y cargar los CSV originales. El backend ofrece la documentación interactiva de rutas mediante Swagger; el frontend se ejecuta con Vite. El repositorio debe incluir código y manifiestos de dependencias, pero excluir secretos, entornos virtuales, carpetas node_modules y archivos temporales. El manual técnico contiene instrucciones para la instalación; el manual de usuario documenta los pasos de interacción con cada pantalla.

Consideraciones de Seguridad y Alcance

La aplicación se ha verificado como prototipo académico en un entorno local. La validación de extensión y tamaño de archivo ayuda a evitar entradas accidentales, pero no sustituye las medidas necesarias para ofrecer un servicio público. Para producción sería necesario agregar autorización, control de acceso por rol, límites de concurrencia, validación más exhaustiva, copias de seguridad, supervisión de los dos motores y conciliación automática cuando una carga queda completa en MongoDB pero incompleta en Neo4j.

Comparativa Técnica NoSQL: MongoDB vs. Neo4j

El desarrollo de USAC Analytics permitió contrastar operativamente los enfoques documental y de grafos ante un mismo volumen masivo de datos académicos:

- **MongoDB (Documental):** Destaca por una velocidad superior en inserciones masivas (bulk write operations) y procesamiento de consultas analíticas sobre grandes volúmenes de datos denormalizados. Es ideal para agregaciones estadísticas directas (conteos, promedios, distribuciones) y filtrado de grandes arreglos mediante su Aggregation Framework.
- **Neo4j (Grafos):** Sobresale en la exploración de relaciones complejas e interconectadas. Consultas como la trazabilidad de un aspirante a través de múltiples intentos, materias y carreras se expresan de forma concisa en Cypher sin requerir costosos JOINs sintéticos. Además, su integración nativa con herramientas visuales permite renderizar subgrafos interactivos altamente explicativos.

Manual Técnico de Instalación y Despliegue

1. Requisitos Previos: Python 3.10+, Node.js 18+, MongoDB v6.0+, Neo4j Community/Enterprise v5.x+.

2. Configuración del Backend (FastAPI): Clonar el repositorio, crear entorno virtual *python -m venv venv* y ejecutar *pip install -r requirements.txt*. Configurar las variables de entorno en el archivo *.env* (MONGO_URI, NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD). Iniciar el servidor mediante *uvicorn main:app --reload*.

3. Configuración del Frontend (React): Acceder al directorio frontend, ejecutar *npm install* para instalar las dependencias (Cytoscape, Recharts, Tailwind CSS) y desplegar el servidor de desarrollo con *npm run dev*.

Manual de Usuario

Dashboard Principal: Muestra métricas globales y gráficos interactivos del porcentaje de aprobación por materia (ver Figura 1).

Modulo de Importación CSV: Carga de archivos delimitados con procesamiento en tiempo real y reporte de rechazos (ver Figura 3 y Figura 4).

Explorador de Grafos: Permite ingresar un correlativo de aspirante para inspeccionar visualmente su red de evaluaciones, asignaciones y carreras relacionadas (ver Figura 2).

Consultas y Búsquedas Avanzadas: Módulos dedicados para ejecutar las consultas estructuradas de MongoDB y Neo4j, aplicando filtros por año, carrera o departamento con exportación de resultados.

Discusión y Recomendaciones

El modelo documental es útil para consultas agregadas por año, materia, carrera e institución, mientras que el grafo hace visible la trayectoria de un aspirante y sus vínculos. La arquitectura integrada evita exigir al usuario final conocimientos de los lenguajes de consulta. Una mejora futura consistiría en contar con un registro inequívoco de filas nuevas, actualizadas y omitidas en cada importación, además de una verificación de duplicidad semántica entre archivos diferentes. También conviene mantener un conjunto pequeño de CSV de prueba automatizada, separado de los datos históricos usados para la presentación final.

Anexo: Evidencias Gráficas

Figura 1. Dashboard de aprobación por materia.



Figura 2. Explorador interactivo: relaciones entre aspirante, evaluaciones, materias y carreras.

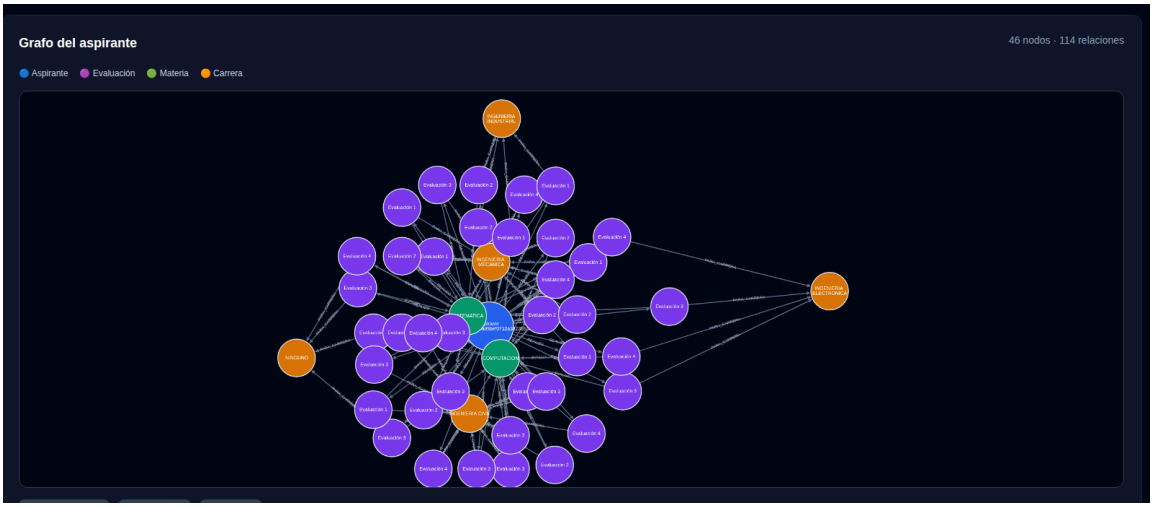


Figura 3. Importación CSV de 2015: 20,277 válidos y sincronizados.

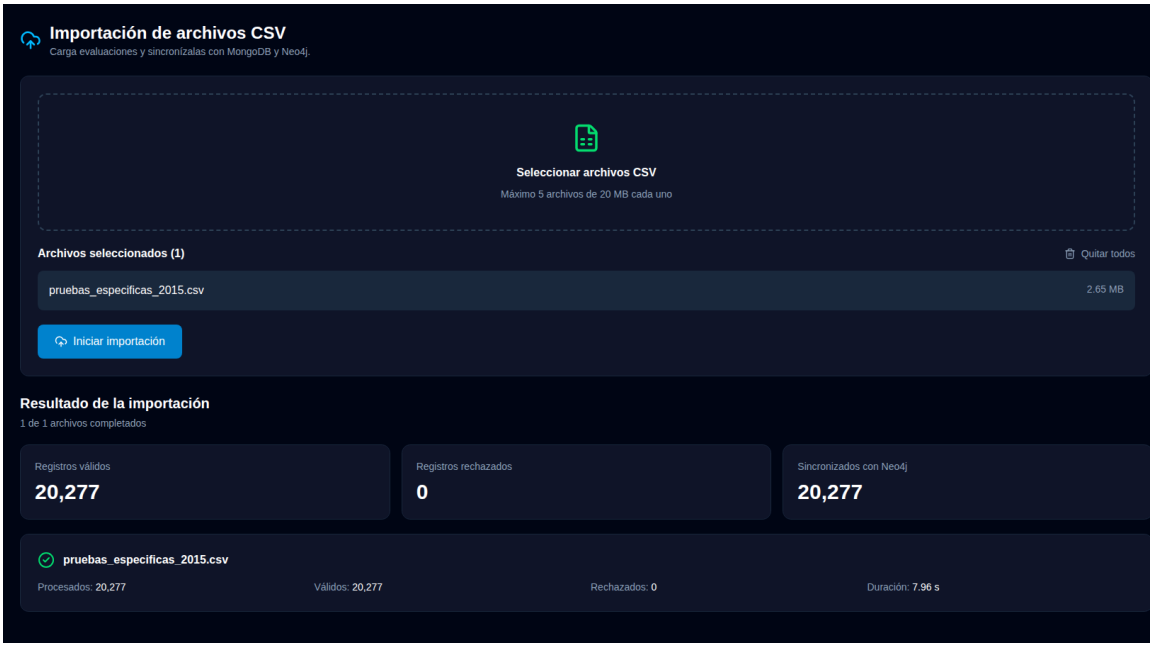


Figura 4. Prueba CSV: tres rechazos y sus motivos visibles.

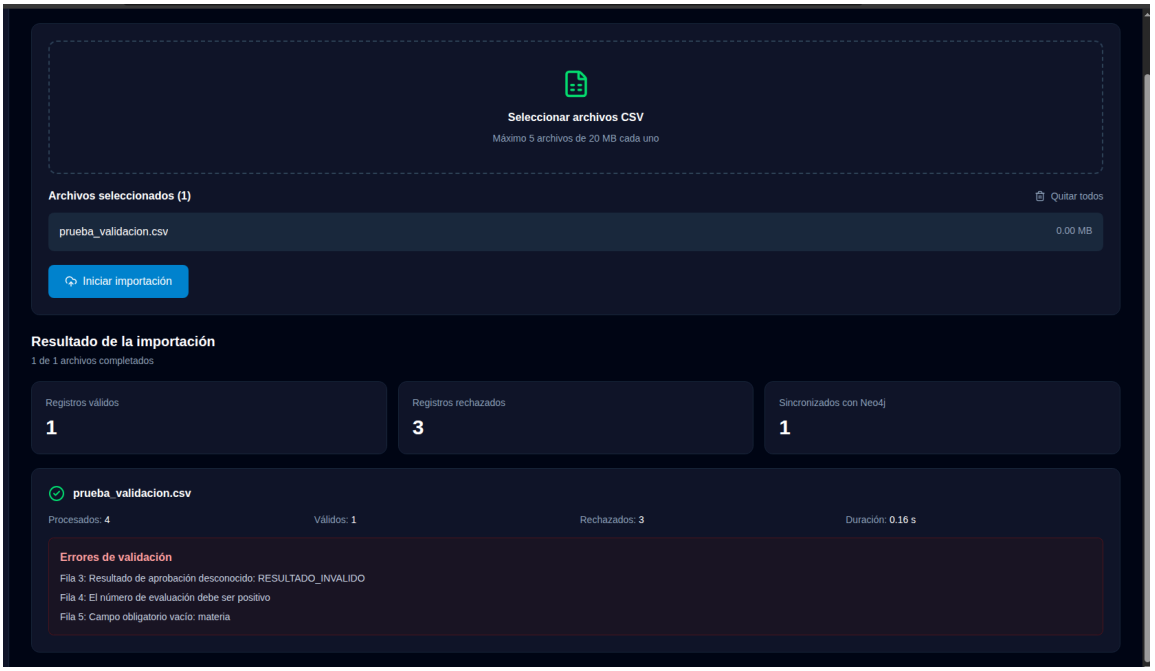


Figura 5. Verificación de integridad en Neo4j: 208,453 relaciones comprobadas.

